

SIMD

a) write a program to perform element-wise addition of two large arrays. Compile using -O0 & -O3 march=native. Measure the execution time & compare the results.

⇒ Algorithm

1. Start
2. Define size N for arrays.
3. Allocate 3 arrays A, B, C .
4. Initialize arrays A & B with values.
5. Record the start time.
6. for $i = 0$ to $(N-1)$
 compute $C[i] = A[i] + B[i]$
7. Record end time.
8. Calculate execution time.
9. Compile once with -O0.
10. Compile again with -O3 -march=native.
11. Compare execution time.
12. Stop.

b) Generate the compiler vectorization report & identify whether the loop is vectorized.

⇒ Algo

1. Start
2. Compile the program using compiler vectorization flags.
3. Generate optimization reports.
4. Check whether the addition loop is marked as vectorized.
5. Check whether the Report compiler remarks.
6. Compare the generated report with the program structure.
7. Stop.

c) Modify the loop to introduce a loop-carried dependency.
Recompile & analyse why vectorization fails.

⇒ Algorithm

1. Start

2. Create arrays A & B.

3. Initialize arrays.

4. Replace with $A[i] = A[i-1] + B[i]$

5. Compile using optimization flags.

6. Generate vectorization report.

7. Observe that each iteration depends on the previous iterations.

8. Conclude that vectorization cannot be applied because of loop-carried dependency.

9. Stop.

d) Implement the same program using OpenMP only, and OpenMP + SIMD optimization. Measure the execution time & compare the results.

⇒ Case 1

1. Start

2. Initialize arrays.

3. Record start time.

4. Parallelize the loop using OpenMP.

5. Perform array addition.

6. Record end time.

7. Calculate execution time.

8. Stop

Case 2

1. Start

2. Initialize arrays.

3. Record start time

4. Apply.
5. Perform vector addition.
6. Record end time.
7. Compute execution time.
8. Compare with:
 - sequential
 - open MP
 - open MP + SIMD
9. Stop.

2) Vector Processing

Vector Program:

```

LV    V1, A           // Load vector A
LV    V2, B           // Load vector B
ADDV  V3, V1, V2       // Vector addition
MULVS V4, V3, F0       // Multiply by scalar
SV    C, V4           // Store result
  
```

Parameters

- Vector length = 64
- Load Startup Latency = 12 cycles, Add Startup Latency = 6 cycles (ADDV)
- Multiply Startup Latency = 7 cycles, Store Startup Latency = 12 cycles (SV)
- Transfer time = 1 cycle/element.

$$\text{Inst. Time} = \text{Startup Latency} + (VL \times \text{Transfer time})$$

a) Assume the processor supports non-chained execution with a single memory port. Cal the total exe. time.

$$\Rightarrow \text{Total} = LV1 + LV2 + ADDV + MULVS + SV$$

$$\rightarrow VL(64) = 76 + 76 + 70 + 71 + 76 = 369 \text{ Cycles}$$

$$\rightarrow VL(128) = 140 + 140 + 134 + 131 + 140 = 685 \text{ Cycles}$$

$$VL = 64$$

- Load : $12 + 64 = 76 \text{ cycles}$
- Mul : $7 + 64 = 71 \text{ cycle}$
- Add : $6 + 64 = 70 \text{ cycle}$
- Store : $12 + 64 = 76 \text{ cycle}$

b) Enable the Vector chaining, while keeping a single memory port. Calculate the total exec. time.

⇒ Two loads cannot occur simultaneously because there is only one memory port.

- LVI : starts at 0, ends at 76
- LV2 : starts at 76 (Port busy), ends at 150
- ADDV : starts at 76 + 12 = 88, ends at 158
- MULVS : starts at 88 + 6 = 94, ends at 165
- SV : starts at 94 + 7 = 101, ends at 101 + 76 = 177 cycles

c) Vector chaining, dual memory ports (for load)

⇒ LVI & LV2 now executed in parallel

- LVI & LV2 : both start at 0, end at 76
- ADDV : starts at 0 + 12 = 12, ends at 80
- MULVS : starts at 12 + 6 = 18, ends at 89
- SV : starts at 18 + 7 = 25, ends at 25 + 76 = 101 cycles

d) Repeat (a) - (c) for VL = 128.

- Per-instruction times : LV = 140, ADDV = 134, MULVS = 135
SV = 140

a) Non-chained, single port :

$$\text{Total} = 140 + 140 + 134 + 135 + 140 = 689 \text{ cycles}$$

b) Chained, single port :

$$\text{• LVI : } 0 - 140$$

$$\text{• LV2 : } 140 - 280$$

• ADDY : Starts $140 + 10 = 150$, ends 286

• MULVS : Starts $150 + 6 = 156$, ends 293

• SV : Starts $158 + 7 = 165$, ends $165 + 140 = 305$ qds

c) chained, dual port :

• LVI & LV2 : both 0 $\rightarrow 140$

• ADDV : Starts $0 + 12 = 12$, ends 146

• MULVS : Starts $12 + 6 = 18$, ends 153

• SV : Starts $18 + 7 = 25$, ends $25 + 140 = 165$ qds

Comparison

	VL = 64	VL = 108
a) Non-chained, single port	369	689
b) chained, single port	177	305
c) chained, dual ports	1010	168